

Inference Engineering Assignment

Part A – Theory

September 9, 2026

Q1. What’s the difference between latency and throughput in model inference, which factor will influence model’s performance more and how?

- **Latency** is the time it takes to process a single request – for example, the time from giving a prompt to getting the output back.
- **Throughput** is how much work the system can handle in total over a given time, such as requests/sec or tokens/sec.
- Both matter, but which one is more important depends on the use case:
 - For chatbot-type applications where an immediate reply is expected, **latency** matters more.
 - For bulk/offline work (e.g. batch scoring), **throughput** matters more, since nobody is waiting on a single result – the goal is to finish the total workload quickly.
- Increasing batch size generally improves throughput (the hardware is used more efficiently), but it slightly increases the latency of an individual request, since that request has to wait for the batch to complete. So this is a trade-off – both cannot be maximized at the same time.

Additional points:

- In LLM serving specifically, latency is often split into **Time-To-First-Token (TTFT)** and **Time-Per-Output-Token (TPOT)**, since these behave differently and matter for different use cases (e.g. TTFT matters for perceived responsiveness).
- Throughput is usually measured as tokens/sec across *all* concurrent users being served, not just one request.
- In production, systems often target a latency service-level objective (e.g. p95 or p99 latency) and try to maximize throughput within that constraint, rather than optimizing either metric in isolation.

Q2. Explain what KV cache is and how it helps with model's generation.

- When a model generates text token by token, each new token needs the attention (K, V vectors) of all the previous tokens.
- Without a cache, all of this would have to be recalculated from scratch at every step, which is wasteful since those values don't change once computed.
- The **KV cache** stores the K, V vectors that have already been calculated. For the next token, only the new token's K, V need to be computed – the older cached values are simply reused.
- This makes generation much faster, especially for long sequences.
- **Trade-off:** the cache takes up RAM/VRAM, and this memory usage grows with longer conversations or more concurrent requests.

Additional points:

- KV cache size scales with sequence length $\times$ number of layers $\times$ number of heads $\times$ head dimension, which is why it becomes a major memory bottleneck for long-context models.
- Techniques like **Multi-Query Attention (MQA)** and **Grouped-Query Attention (GQA)** reduce KV cache size by sharing keys/values across multiple attention heads.
- Serving systems such as vLLM use **PagedAttention** to manage KV cache memory more efficiently (similar to OS-style virtual memory paging), which reduces fragmentation and lets more requests be served at once.

Q3. What is quantization, and what's the trade-off between int8/int4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed.

- **Quantization** means converting a model's weights from high precision (FP16) to lower precision (such as INT8/INT4).
- **Benefit:** the model becomes smaller and inference can be faster, since integer operations are cheaper than floating-point operations on a CPU.
- **Trade-off:** reducing precision causes some information loss, so accuracy can drop slightly. INT8 usually causes a smaller drop, while INT4 causes a bigger drop, because the representable range becomes even smaller.

Quantization: $x_q = \text{round}(x/S) + Z$

Dequantization: $x = S \times (x_q - Z)$

Example: weight $x = 0.72$ (FP16), scale $S = 0.0078$, zero-point $Z = 0$

$$x_q = \text{round}(x/S)$$

$$x_q = \text{round}(0.72/0.0078) = 92$$

So this weight will be stored as 92 in INT8. If we want it back in float form, we dequantize it:

$$x = S \times (x_q - Z) = 0.0078 \times 92 \approx 0.718.$$

The small difference here between 0.72 and 0.718 is the **quantization error**. Fewer bits means more compression and speed, but also a higher risk of error.

Quantization can also be done from FP16 to INT8 using the same formula. For example, if the same weight $x = 0.72$ is stored in FP16, and it is quantized to INT8 using the same scale $S = 0.0078$ and zero-point $Z = 0$:

$$x_q = \text{round}(0.72/0.0078) = 92$$

The result comes out the same (92), because the quantization formula only depends on the actual value, not on the source precision (whether it was FP32 or FP16). It's just that if you start from FP16, the model was already smaller, so the overall memory saving compared to going $\text{FP32} \rightarrow \text{INT8}$ will be less ($\text{FP16} \rightarrow \text{INT8}$ gives only 2x compression, while $\text{FP32} \rightarrow \text{INT8}$ gives 4x compression).

Additional points:

- **Static quantization** uses a calibration dataset to fix the scale/zero-point before deployment, while **dynamic quantization** (e.g. `torch.quantization.quantize_dynamic`) computes the scale/zero-point on the fly at runtime.
- **Per-channel quantization** (a separate scale per output channel) usually preserves accuracy better than **per-tensor quantization** (a single scale for the whole tensor).
- Very low-bit quantization (like INT4) often needs extra techniques – e.g. **GPTQ** or **AWQ** – to keep accuracy acceptable, since the value range available is very limited.

Q4. Name one real inference serving framework (like vLLM or TensorRT-LLM) and briefly describe one optimization it does.

One real inference serving framework is vLLM. It is used to run and serve Large Language Models efficiently. One important optimization in vLLM is PagedAttention. It manages the GPU memory used during text generation in a better way. This reduces memory wastage and allows the system to handle more requests at the same time. Because of this, vLLM can improve the overall performance of LLM inference.

Another useful feature of vLLM is continuous batching. It can combine requests that arrive at different times and process them efficiently instead of waiting for a fixed batch to finish. This helps the GPU stay busy and improves the speed and overall throughput of the model. Therefore, vLLM is useful for serving AI models when many users are sending requests at the same time.

Additional points:

- **TensorRT-LLM** (NVIDIA) is another popular framework; it performs optimizations such as kernel fusion and support for various quantization formats to speed up inference on NVIDIA GPUs.
- vLLM also supports **tensor parallelism** across multiple GPUs and several quantization methods, extending beyond just PagedAttention and continuous batching.